
Ministerul Educației și cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 3

Empirical analysis of algorithms: Depth First Search
(DFS), Breadth First Search(BFS)

Elaborated:

std. gr. FAF-231

Nicolae MARGA

Verified:

asist. univ.

Cristofor FIȘTIC

Chișinău – 2025

Contents

1	Algorithm Analysis	3
1.1	Objective	3
1.2	Tasks	3
1.3	Theoretical Notes	3
1.3.1	Graph Classification by Connectivity and Density	3
1.3.2	Density Classification	4
1.4	Introduction	4
1.4.1	Included Graph Types	4
2	Implementation	5
2.1	Environment	5
2.2	Traversal Functions	5
2.3	Measurement Approach	6
3	Results	7
3.1	Execution Time Comparison	7
3.2	Ratio Analysis	7
3.3	Heatmaps	8
3.4	Comparison	9
4	Observations	10
5	Conclusion	10
6	Repository	10

1 Algorithm Analysis

1.1 Objective

Study and analyze Depth-First Search (DFS) and Breadth-First Search (BFS) graph traversal algorithms. Perform empirical analysis

1.2 Tasks

- Implement DFS and BFS algorithms.
- Establish the properties of the input data against which the analysis is performed
- Choose metrics for comparing algorithms
- Perform empirical analysis of the proposed algorithms
- Make a graphical presentation of the data obtained
- Make a conclusion on the work done

1.3 Theoretical Notes

Graphs are mathematical structures used to model relationships between objects, consisting of nodes (vertices) connected by edges. They help to represent networks, hierarchies, and interconnected systems in computer science, with a lot of wide applications ranging from social networks to route planning.

1.3.1 Graph Classification by Connectivity and Density

Graphs can be classified in several ways. Based on edge density, we distinguish the following types: Trees, which are connected acyclic graphs; Sparse graphs, which have relatively few edges compared to the number of nodes; Dense graphs, which have a higher number of edges; and Complete graphs, where all possible edges between vertices are present.

Regarding edge directionality, graphs can be classified as Directed graphs, in which edges have a specific orientation, and Undirected graphs, where edges do not have a direction.

Based on connectivity, graphs are either Connected graphs, which consist of a single connected component, or Disconnected graphs, which contain multiple components.

1.3.2 Density Classification

A connected graph can be further categorized according to its edge density. Edge density significantly influences the structural properties and the performance of algorithms operating on graphs.

In undirected graphs, the minimum number of edges in a connected graph (a tree) is $n - 1$, while the maximum number of edges in a complete graph is $\frac{n(n-1)}{2}$. Sparse graphs have far fewer than this upper bound, and dense graphs have edge counts approaching it.

1.4 Introduction

Graph traversal algorithms are important for exploring or searching graph structures.

Depth-First Search searches as far as possible along each branch before it backtracks. It uses a stack and is best for tasks like topological sorting or detecting cycles.

Breadth-First Search searches all neighbor nodes at the present depth before moving to nodes at the next depth level. It makes use of a queue and is best for finding shortest paths in unweighted graphs.

Time complexity of both algorithms are of complexity $\mathcal{O}(V + E)$ where V is the number of vertices and E is the number of edges.

1.4.1 Included Graph Types

Sparse Graph: A graph with relatively few edges compared to the number of possible edges. These graphs have low edge density and often represent systems with minimal connectivity, such as social networks with few interactions.

Dense Graph: A graph where the number of edges is close to the maximum possible. In such graphs, most pairs of vertices are connected, making them highly interconnected.

Grid Graph: A graph structured like a 2D grid, where each node is typically connected to its adjacent nodes (up, down, left, right). Grid graphs are common in image processing and pathfinding algorithms.

Cycle Graph: A graph that forms a single cycle, meaning each vertex is connected in a closed loop with exactly two neighbors. These are useful for modeling circular structures or processes.

Star Graph: A tree with one central node connected directly to all other nodes (leaves), and no other edges. It models hub-and-spoke networks like centralized communication systems.

Complete Graph: A graph in which every pair of distinct vertices is connected by a unique edge. These graphs are maximally connected and often used in theoretical analysis for worst-case scenarios.

2 Implementation

2.1 Environment

The project was implemented in Python using the following libraries:

- `networkx` for graph generation and manipulation,
- `matplotlib` and `seaborn` for visualization,
- `numpy` for numerical operations,
- `time` for execution time measurement.

2.2 Traversal Functions

Traversal implementations were based on iterative logic with randomized neighbor orderings to minimize traversal biases.

```
def dfs(G, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            stack.extend(reversed(list(G.neighbors(node))))
    return visited

def bfs(G, start):
    visited = set([start])
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbor in G.neighbors(node):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return visited
```

2.3 Measurement Approach

Each traversal was timed using multiple trials to compute an average execution time and standard deviation. Ratios were calculated to compare performance.

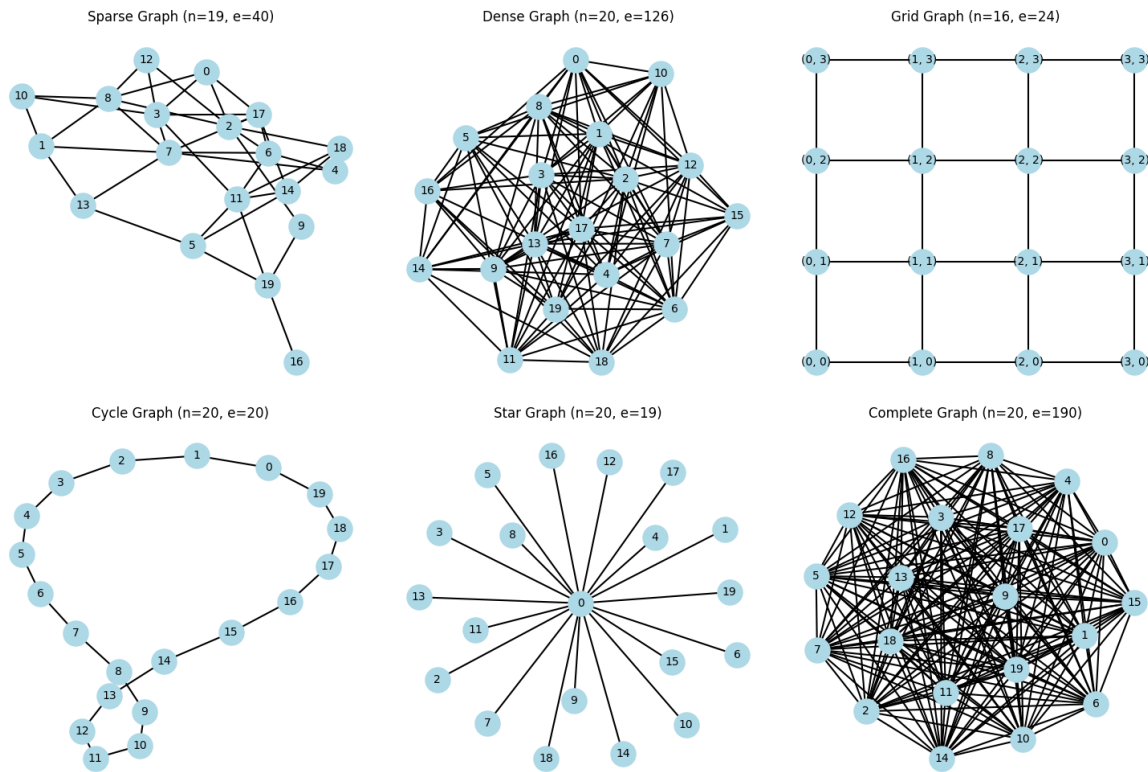


Figure 1: Graphs examples used for BFS and DFS.

3 Results

3.1 Execution Time Comparison

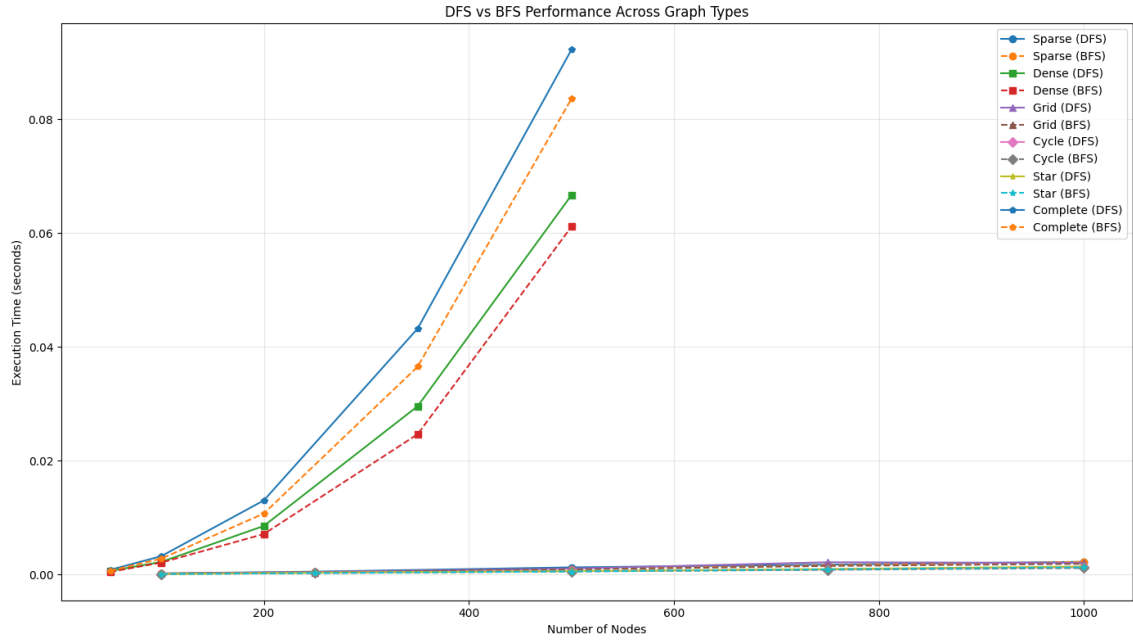


Figure 2: DFS and BFS execution times across different graph types.

3.2 Ratio Analysis

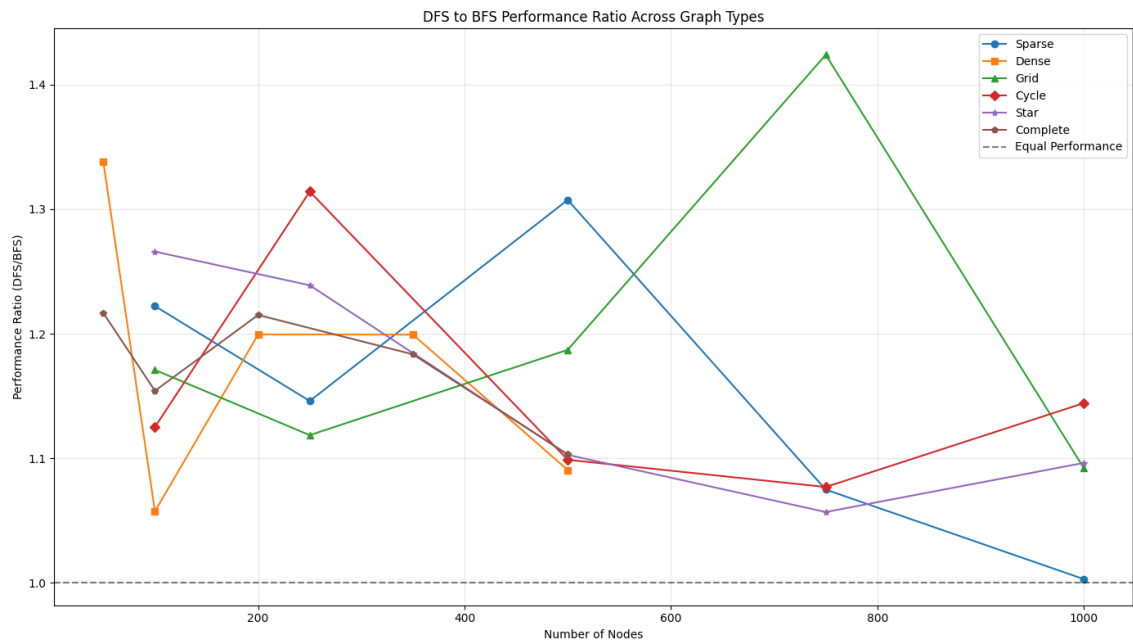


Figure 3: DFS-to-BFS time ratio. Values > 1 indicate BFS is faster.

3.3 Heatmaps

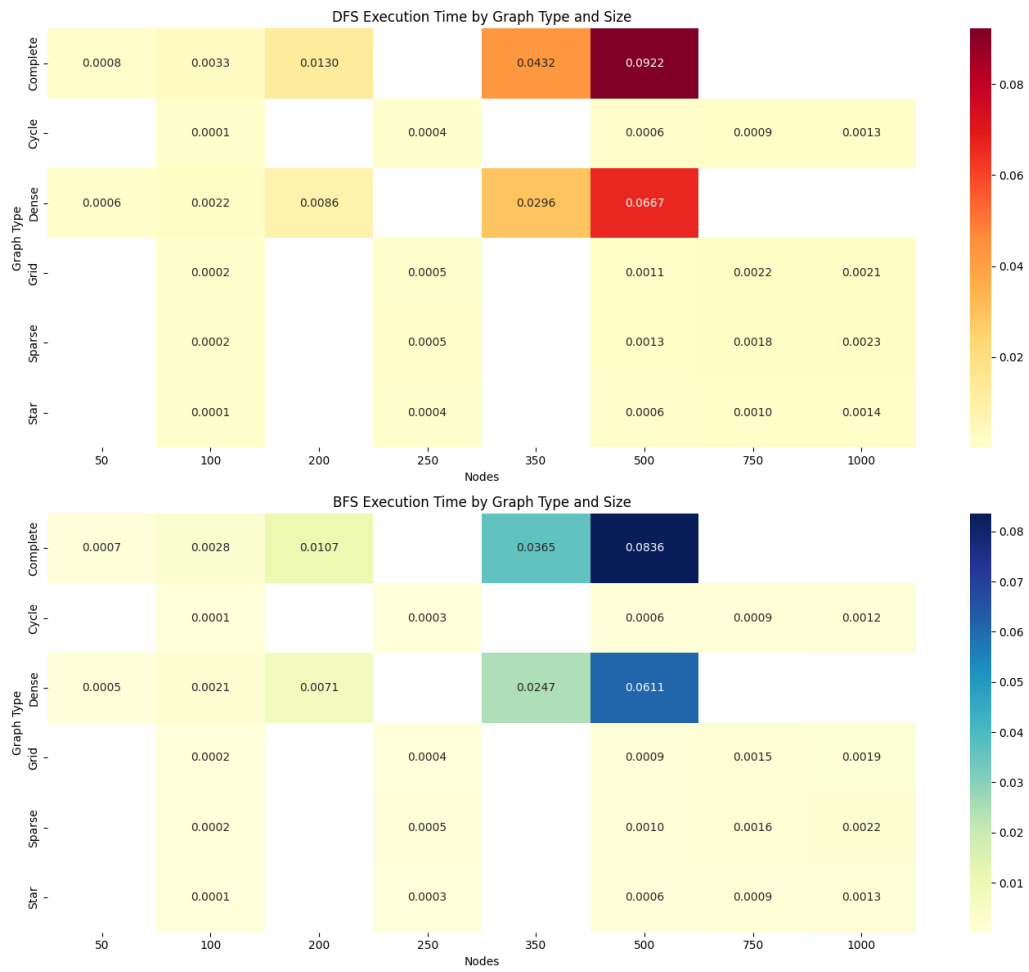


Figure 4: Execution time heatmaps for BFS and DFS.

3.4 Comparison

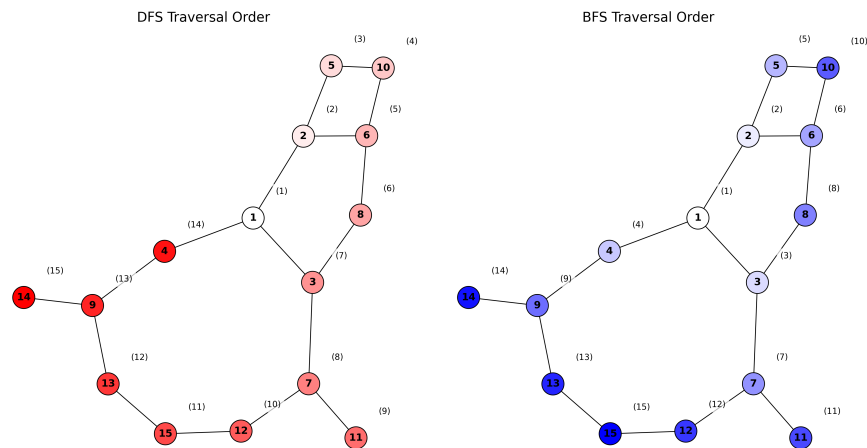


Figure 5: Node traversal comparisons for DFS (left) and BFS (right).

DFS vs BFS Traversal Order

Step	DFS Node	BFS Node
1	1	1
2	2	2
3	5	3
4	10	4
5	6	5
6	8	6
7	3	7
8	7	8
9	11	9
10	12	10
11	15	11
12	13	12
13	9	13
14	4	14
15	14	15

Figure 6: Node traversal table for DFS (left) and BFS (right).

4 Observations

- On **sparse graphs**, DFS and BFS performed similarly.
- On **dense and complete graphs**, BFS consistently outperformed DFS due to queue efficiency.
- **Star graphs** favored BFS due to the central hub layout.
- Grid and cycle graphs showed moderate differences in traversal time.

5 Conclusion

The traversal performance of DFS and BFS is largely dependent on the graph's structure. BFS showed better performance on most types due to its level-order nature, while DFS's depth-first approach became inefficient in highly connected scenarios. These insights can inform algorithm choice in practical applications such as search engines, network analysis, and AI pathfinding.

6 Repository

The source code for this laboratory work can be found here:

https://github.com/xnikug/AA_Labs/tree/lab3